Day 03: Control Statements, Loops & Arrays

Java Day3-notes

Control Statement:

while Loop in Java

The while loop is a control statement used to execute a block of code repeatedly as long as a given condition is true.

Syntax

```
while (condition) {
    // statements
}
.
```

Example 1: Print Numbers from 1 to 5

```
public class WhileDemo {
    public static void main(String[] args) {
```

```
        int i = 1;
```

```
        while (i <= 5) {
            System.out.println(i);
            i++;
        }
    }
}
Output
1
2
3
4
5
```

Example 2: Print Even Numbers

Write a Java program to print even numbers from 2 to 10.

```
public class WhileDemo {
    public static void main(String[] args) {
```

```
        int i = 2;
```

```
        while (i <= 10) {
            System.out.println(i);
            i = i + 2;
        }
    }
}
Example 3: Sum of Numbers
Write a Java program to find the sum of numbers from 1 to 5.
public class WhileDemo {
    public static void main(String[] args) {
```

```
        int i = 1;
        int sum = 0;
```

```
        while (i <= 5) {
            sum = sum + i;
            i++;
        }
```

System.out.println("Sum = " + sum);

}

}

Output

Sum = 15

Working

i sum

1 1

2 3

3 6

4 10

5 15

Example 4: Reverse a Number

A common real programming problem using while is reversing a number.

For example:

Input : 1234

Output : 4321

```
public class WhileDemo {
    public static void main(String[] args) {
```

```
        int num = 1234;
        int reverse = 0;
```

```
        while (num > 0) {
```

```
            int digit = num % 10;
```

reverse = reverse * 10 + digit;

num = num / 10;

}

System.out.println("Reverse = " + reverse);

}

}

Working

For 1234:

digit = 1234 % 10 = 4

reverse = 0 * 10 + 4 = 4

digit = 123 % 10 = 3

reverse = 4 * 10 + 3 = 43

digit = 12 % 10 = 2

reverse = 43 * 10 + 2 = 432

digit = 1 % 10 = 1

reverse = 432 * 10 + 1 = 4321

Finally:

Reverse = 4321

Sum of Digits Using while Loop

Problem

Write a Java program to read an integer and find the sum of its digits using a while loop.

Example:

Input: 12345

Output: Sum of digits = 15

Program

```
import java.util.Scanner;
```

```
public class SumOfDigits {
```

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter a number: ");

```
        int num = sc.nextInt();
```

```
        int sum = 0;
```

```
        while (num > 0) {
```

```
            int digit = num % 10;
```

sum = sum + digit;

num = num / 10;

}

System.out.println("Sum of digits = " + sum);

}

}

So the basic logic is:

Extract last digit → Add to sum → Remove last digit → Repeat

for Loop in Java

The for loop is a control statement used to execute a block of code repeatedly.

It is especially useful when we know how many times we want to execute a statement.

Syntax

```
for (initialization; condition; update) {
    // statements
}
```

```
The three parts are:
Initialization – executed only once at the beginning.
Condition – checked before every iteration.
Update – executed after each iteration.
Flow
Initialization
      ↓
  Condition
   ■     ■
 true    false
  ↓        ↓
Execute    Exit
  ↓
Update
  ↓
Condition
```

Example 1: Print Numbers from 1 to 5

```
public class ForDemo {
    public static void main(String[] args) {
```

```
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}
```

```
Output
1
2
3
4
5
```

Example 2: Multiplication Table

Write a Java program to read a number and print its multiplication table from 1 to 10.

```
import java.util.Scanner;
```

```
public class Table {
    public static void main(String[] args) {
```

Scanner sc = new Scanner(System.in);

System.out.print("Enter a number: ");

```
        int num = sc.nextInt();
```

```
        for (int i = 1; i <= 10; i++) {
            System.out.println(num + " x " + i + " = " + (num * i));
        }
    }
}
Input
Enter a number: 5
Output
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

switch-case in Java

The switch-case statement is a decision-making control statement used when we want to choose one option from multiple possible options.

It is useful when we are comparing one expression against multiple fixed values.

Syntax

Example 2: Simple Calculator

Write a Java program to read two numbers and an operator and perform the selected operation using switch-case.

```
import java.util.Scanner;
```

```
public class Calculator {
```

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter first number: ");

double a = sc.nextDouble();

System.out.print("Enter second number: ");

double b = sc.nextDouble();

System.out.print("Enter operator (+, -, *, /): ");

char op = sc.next().charAt(0);

```
        switch (op) {
```

case '+':

System.out.println("Result = " + (a + b));

```
break;
case '-':
System.out.println("Result = " + (a - b));
break;
case '*':
System.out.println("Result = " + (a * b));
break;
case '/':
if (b != 0)
System.out.println("Result = " + (a / b));
else
System.out.println("Cannot divide by zero");
break;
default:
System.out.println("Invalid operator");
}
}
}
```

Simple Menu-Driven Program

```
import java.util.Scanner;
public class MenuDemo {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
        int choice;
        while (true) {
System.out.println("\n--- MENU ---");
System.out.println("1. Addition");
System.out.println("2. Subtraction");
System.out.println("3. Multiplication");
System.out.println("4. Division");
System.out.println("5. Exit");
System.out.print("Enter your choice: ");
```

choice = sc.nextInt();

```
            switch (choice) {
```

case 1:

System.out.print("Enter two numbers: ");

```
                    int a = sc.nextInt();
                    int b = sc.nextInt();
                    System.out.println("Result = " + (a + b));
                    break;
```

case 2:

System.out.print("Enter two numbers: ");

a = sc.nextInt();

b = sc.nextInt();

System.out.println("Result = " + (a - b));

break;

case 3:

System.out.print("Enter two numbers: ");

a = sc.nextInt();

b = sc.nextInt();

System.out.println("Result = " + (a * b));

break;

case 4:

System.out.print("Enter two numbers: ");

a = sc.nextInt();

b = sc.nextInt();

if (b != 0)

System.out.println("Result = " + (a / b));

else

System.out.println("Cannot divide by zero");

break;

case 5:

System.out.println("Thank you!");

System.exit(0);

default:

System.out.println("Invalid choice");

}

}

}

}

Array in Java

An array is a collection of multiple values of the same data type stored under a single variable name.

For example, instead of creating:

```
int mark1 = 80;
int mark2 = 75;
int mark3 = 90;
we can use an array:
int[] marks = {80, 75, 90};
Here, marks stores three integer values.
Important Points
An array stores values of the same data type.
Array indexing starts from 0.
The size of an array is fixed after creation.
Each element is accessed using its index.
For example:
Index:    0    1    2    3
         -------------------
Array:   10   20   30   40
So:
marks[0]   // 10
marks[2]   // 30
```

1. Integer Array Example

Problem

Write a Java program to read 5 integer values into an array and display all the elements.

```
import java.util.Scanner;
```

```
public class IntegerArrayDemo {
```

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

int[] numbers = new int[5];

// Reading values

```
        for (int i = 0; i < 5; i++) {
            System.out.print("Enter number " + (i + 1) + ": ");
            numbers[i] = sc.nextInt();
        }
```

// Displaying values

System.out.println("Array elements:");

```
        for (int i = 0; i < 5; i++) {
            System.out.println(numbers[i]);
        }
    }
}
Example Output
Enter number 1: 10
Enter number 2: 20
Enter number 3: 30
Enter number 4: 40
Enter number 5: 50
```

Array elements:

10

20

30

40

50

Understanding

int[] numbers = new int[5];

creates an integer array of size 5.

numbers[i] = sc.nextInt();

stores the input value at index i.

For 5 elements:

numbers[0]

numbers[1]

numbers[2]

numbers[3]

numbers[4]

Notice that the last index is 4, not 5.

2. String Array Example

A String array is used to store multiple strings.

Problem

Write a Java program to read 5 student names into an array and display all the names.

```
import java.util.Scanner;
```

```
public class StringArrayDemo {
```

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

```
        String[] names = new String[5];
```

// Reading names

```
        for (int i = 0; i < 5; i++) {
            System.out.print("Enter student name " + (i + 1) + ": ");
            names[i] = sc.nextLine();
        }
```

// Displaying names

System.out.println("\nStudent Names:");

```
        for (int i = 0; i < 5; i++) {
            System.out.println(names[i]);
        }
    }
}
Example Output
Enter student name 1: Rahul
Enter student name 2: Anu
Enter student name 3: Kiran
Enter student name 4: Meera
Enter student name 5: Arjun
```

Student Names:

Rahul

Anu

Kiran

Meera

Arjun

Another Way to Create an Array

We can directly initialize an array with values.

Integer Array

int[] numbers = {10, 20, 30, 40, 50};

String Array

```
String[] names = {"Rahul", "Anu", "Kiran", "Meera"};
Accessing Elements
System.out.println(numbers[0]);
System.out.println(names[2]);
Output:
10
Kiran
Remember
Array
  ↓
Multiple values
  ↓
Same data type
  ↓
Index starts from 0
  ↓
Access using array[index]
A very common combination in Java is array + for loop, because the loop can be used to read, display, search, a
```